BYPASS RATE LIMIT Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Rate-Limit Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use only approved test accounts and a program-approved request budget. Do not brute-force credentials, evade protections on third-party services, or generate traffic that could degrade availability.*

■ Executive Summary

Rate limiting reduces automated abuse of login, password-reset, MFA, confirmation-code, and registration workflows. A robust implementation must count requests consistently across relevant routes and identity signals without trusting attacker-controlled headers or allowing trivial session changes to reset the limit.

■ Core Testing Methodology

1. **Identify sensitive endpoints.** Review login, password reset, MFA, confirmation-code, and sign-up flows.
2. **Read the program rules.** Establish the maximum approved request volume and stop conditions before testing.
3. **Establish a baseline.** Send a small number of requests with a controlled account and record response behavior.
4. **Check limit enforcement.** Confirm that the server throttles or blocks requests at the documented threshold.
5. **Review identifier consistency.** Verify that changing a session or cookie does not trivially reset protection where the policy is meant to cover the account or source.
6. **Review proxy configuration.** Ensure client-supplied IP headers cannot arbitrarily change the rate-limit identity.
7. **Review normalization.** In a lab, check whether the same logical identifier is counted consistently when represented with harmless whitespace or encoding variations.
8. **Check concurrent requests carefully.** Use a low-volume controlled test to verify that simultaneous requests cannot exceed a critical one-use limit.
9. **Verify recovery behavior.** Confirm that legitimate users can recover access through documented processes without weakening the control.
10. **Document and retest.** Record the limit, request count, response, and account state without exposing real user data.

■ High-Value Bug Bounty Tips & Edge Cases

- **Where to look:** The source lists login, password reset, 2FA, confirmation codes, and sign-up.
- **Null/encoding edge cases:** The source mentions null-character variants. Treat these as input-normalization tests in a local lab; do not use them to evade a live service's controls.
- **Cookie changes:** Rate limits should not depend solely on a freely replaceable client cookie.
- **Host header:** Host-related values must not be used as a user identity or trusted rate-limit key without a secure design.
- **Forwarding headers:** `X-Forwarded-For` and similar headers should be set by trusted proxies and not accepted arbitrarily from the public internet.
- **Distributed limits:** Consider per-account, per-IP/network, per-device or risk-based controls, with privacy and false-positive trade-offs.
- **Source limitation:** The two-page source is a brief checklist and does not define a particular application's thresholds or demonstrate a working bypass.

■■ Educational Checklists & Safe Examples

```
Endpoint: [In-Scope-Login-or-Recovery-Endpoint]
Test account: [Controlled-Test-Account]
Approved request budget: [Program-Defined-Limit]
```

```
Observed threshold: [Count or Not Determined]
Throttle response: [Redacted]
Changing session resets limit: [Yes/No/Not Tested]
Forwarding headers trusted from clients: [Yes/No/Not Tested]
```

- [ ] Sensitive endpoints have documented limits.
- [ ] Limits are enforced server-side.
- [ ] Account-level limits cannot be reset by changing a cookie.
- [ ] Only trusted proxy infrastructure determines client IP.
- [ ] Concurrent requests cannot bypass one-use or strict limits.
- [ ] Monitoring and user-safe recovery are in place.

■■ Mitigation & Remediation

- Apply layered limits by account, source network, endpoint, and risk signal.
- Trust client IP information only when supplied by configured reverse proxies.
- Normalize identifiers before counting requests and reject malformed values consistently.
- Use atomic counters or equivalent concurrency-safe controls.
- Add progressive delays, alerts, and risk-based challenges where appropriate.
- Avoid revealing exact internal thresholds unnecessarily, while providing usable error and recovery paths.
- Monitor abuse and test limits in staging before deployment.

■ Source Notes

Based on the supplied two-page *BYPASS RATE_LIMIT.pdf*. Page 1 lists sensitive authentication workflows, null-character-related input variations, host-header changes, cookie changes, and forwarding headers. Page 2 shows additional forwarding-header examples. The source does not provide verified results for a particular application.

Educational use only. Stay within scope and the approved request budget.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.